

Algoritmi di Parsing: Bottom-up Parsing

Slides based on material
by Ras Bodik available at
<http://inst.eecs.berkeley.edu/~cs164/fa04>

Welcome to the running example

- we'll build a parser for this grammar:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * \text{int} \mid \text{int}$$

(non ambiguous grammar with no useless variables)

- see, the grammar is

- left-recursive

- not left-factored

- ... and our parser won't mind!

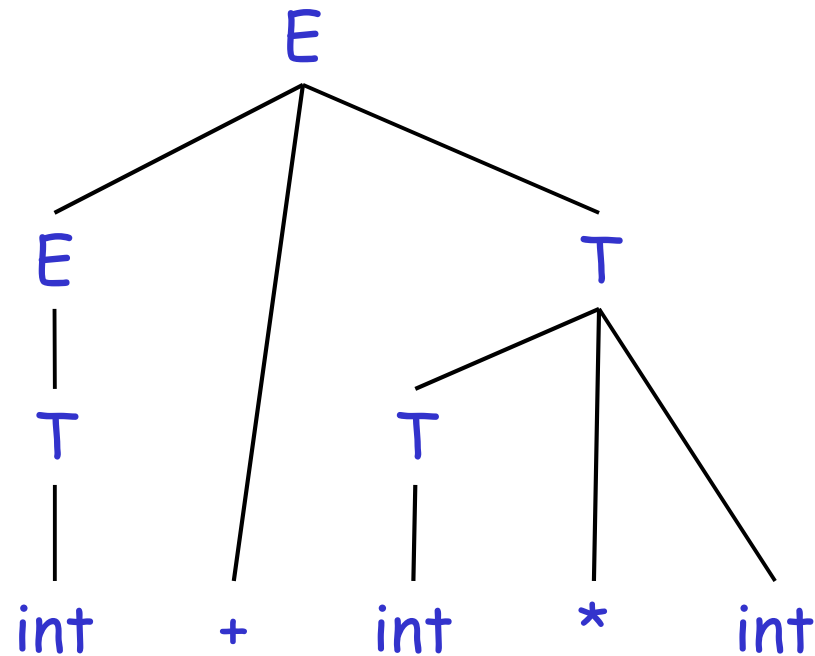
Gli algoritmi di parsing Bottom-Up (in particolare quelli LR) sono molto più potenti dei parser Top-Down (LL). I parser LR possono gestire grammatiche ricorsive a sinistra e molte grammatiche che non sono fattorizzate a sinistra senza necessità di riscriverle, rendendole ideali per la sintassi dei linguaggi di programmazione reali.

Example input, parse tree

- input:

int + int * int

- its parse tree:



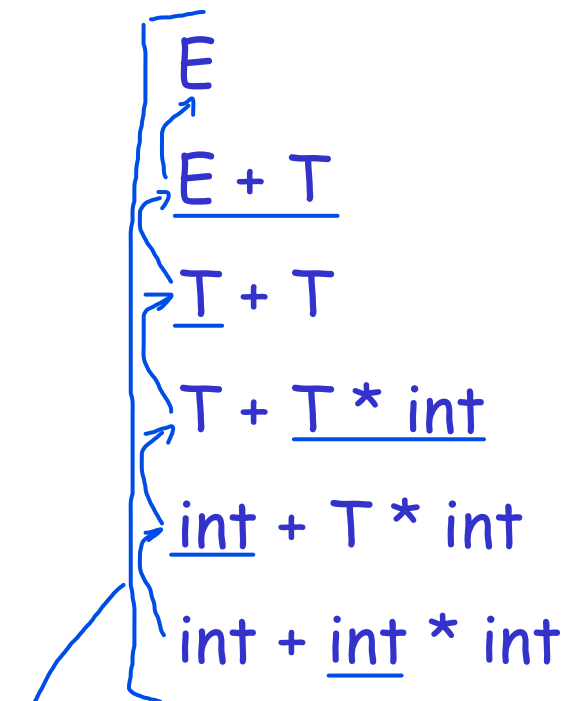
1

Chaotic bottom-up parsing

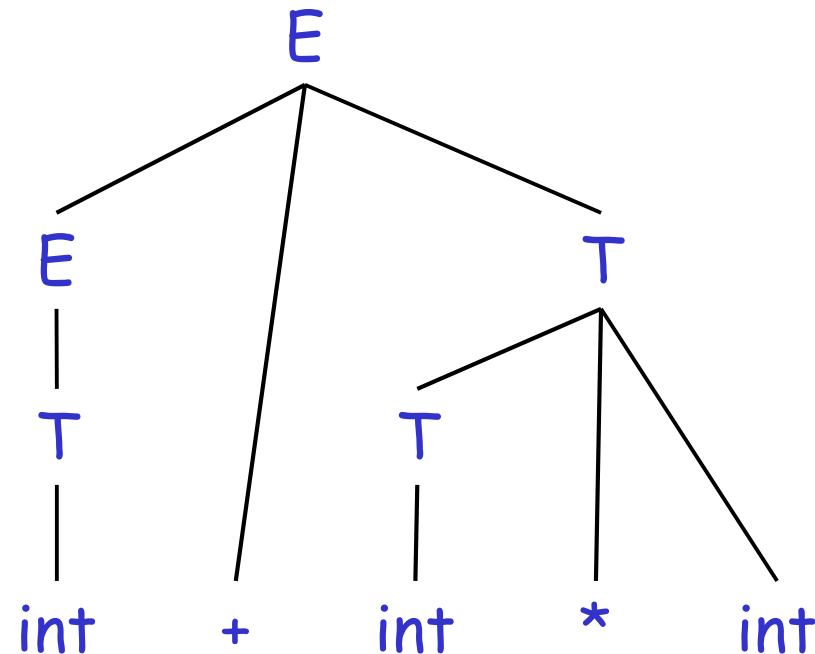
Derivazione al Contrario

Key idea: build the derivation in reverse

(Riduzione da Corpo della
Produzione a Testa della Produzione)



Guardo al punto in cui
sono arrivato ed
applico una produzione
al contrario (scelta a
caso).



Algo:
Guarda dove vuoi nella stringa (int + int * int). Trova
il corpo di una produzione presente nella stringa.
Riduci la stringa trovata alla testa. Continua fino a
che ho il Parse Tree (cioè arrivo alla radice)

Chaotic bottom-up parsing

- The algorithm:

1. fissa con lo sguardo la stringa di input s stare at the input string s

- feel free to look anywhere in the string

2. find in s a right-hand side r of a production $N \rightarrow r$ (trova una produzione a cui ridurre alla testa)

- ex.: found int for a production $T \rightarrow \text{int}$

3. reduce the found string r into its non-terminal N (Riduzione)

- ex.: replace int with T

4. if all string reduced to start non-terminal

- we're done, string is parsed, we got a parse tree

5. otherwise continue in step 1

Don't celebrate yet!

NON SEMPRE FUNZIONA L'ALGORITMO (funziona solo se si ha fortuna di scegliere la giusta produzione da ridurre)

- not guaranteed to parse a correct string

(Posso arrivare in un punto bloccante)

- is this surprising?

- example:

and we are stuck

$E + E * E$

$E + E * \underline{T}$

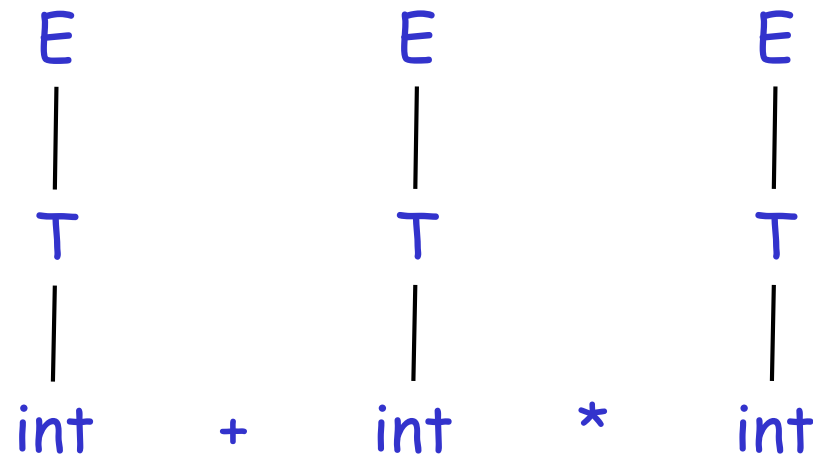
$E + E * \underline{\text{int}}$

$\underline{T} + E * \text{int}$

$\underline{\text{int}} + E * \text{int}$

$\text{int} + \underline{T} * \text{int}$

$\text{int} + \underline{\text{int}} * \text{int}$



Lesson from chaotic parser

- Lesson:

- if you're lucky in selecting the string to reduce next, then you will successfully parse the string

→ Come posso superare il problema?

How to “beat the odds”?

- that is, how to find a lucky sequence of reductions that gives us a derivation of the input string?
- use non-determinism! (Prova tutte le possibili combinazioni di Riduzioni)

2

Se ci sono k modi per ridurre la stringa attuale, crea k copie identiche del parser, dove applicare una delle k riduzioni in ciascun copia del parser

bottom-up

Non-deterministic chaotic parser

Inefficiente

The algorithm:

1. find in input all strings that can be reduced
 - assume there are k of them (cioé su quella stringa attuale, posso effettuare k riduzioni diverse)
2. create k copies of the (partially reduced) input
 - it's like spawning k identical instances of the parser
3. in each instance, perform one of k reductions
 - and then go to step 1, advancing and further spawning all parser instances
4. stop when at least one parser instance reduced the string to start non-terminal (Termina quando una copia genera il Parse Tree)

Properties of the n.d. chaotic parser

Claim:

- the input will be parsed by (at least) one parser instance

But:

- ¹ exponential blowup: $k * k * k * \dots * k$ parser copies

Also:

- ² Multiple (usually many) instances of the parser produce the correct parse tree (due to multiple corresponding derivations). This is wasteful.

Overview

- ① • Chaotic bottom-up parser
 - it will give us the parse tree, but only if it's lucky
- ② • Non-deterministic chaotic parser
 - creates many parser instances to make sure at least one builds the parse tree for the string
 - an instance either builds the parse tree or gets stuck
- ③ • Non-deterministic LR parser (next)
 - restrict where a reduction can be made
 - as a result, fewer instances necessary

Non-deterministic LR parser

→ Vogliamo avere un unico successo e la possibilità di clonare i parser (mantenere il non-determinismo)

- What we want:

- create multiple parser instances

- to find the lucky sequence of reductions

but the parse tree is found by at most ^{al massimo} one instance

- zero if the input has syntax error

② 1:2
→ Deve risolvere i due problemi dell'algoritmo precedente (Esplosione delle copie del parser e Multiple soluzioni)

Riduzione (Reduce): rimpiazzo la cima dello stack (scelgo la produzione)
Shift: metto nuova stringa in cima allo stack per avere più riduzioni possibili

Two simple rules to restrict # of instances

Shift (left to right) and Reduce (right to left)

1. split the input in two parts:

- **right:** unexamined by parser
- **left:** in the parser (we'll do the reductions here)

Divido l'input in due parti: il parser esamina solo la parte a sinistra della freccia e la riduzione ammessa è quella vicina al punto di divisione delle due parti

$\text{int} \triangleright + \text{int} * \text{int}$ after reduction: $T \triangleright + \text{int} * \text{int}$

2. reductions allowed only on part adjacent to split

Derivazione canonica a destra (quindi l'algoritmo trova un'unica derivazione, se ha successo)

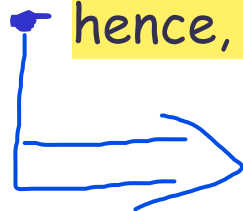
allowed: $T + \underline{\text{int}} \triangleright * \text{int}$

after reduction: $T + \underline{T} \triangleright * \text{int}$

not allowed: $\underline{\text{int}} + \text{int} \triangleright * \text{int}$

after reduction: $\underline{T} + \text{int} \triangleright * \text{int}$

hence, left part of string can be kept on the stack



Si può vedere come uno stack con cima a dx per capire quali riduzioni sono ammesse

Wait a minute!

Queste restrizioni non sono fatalmente severe?

Aren't these restrictions fatally severe?

nessuna istanza riesce
ad analizzare l'input

← the doubt: no instance succeeds to parse the input

No. Recall:

⇒ one parse tree corresponds to multiple derivations

- in n.d. chaotic parser, the instances that build the same parse tree each follow a different derivation

Le caratteristiche del parsing LR non impediscono di trovare il Parse Tree quando l'input è valido, perché:

- Se la stringa è valida, esiste un Parse Tree unico.
- Un Parse Tree unico implica una sola derivazione canonica a destra.
- L'algoritmo LR è progettato per trovare l'inverso di questa derivazione canonica a destra unica attraverso le sue operazioni di Shift e Reduce.

Wait a minute! (cont)

recall: two interesting derivations

- left-most derivation, right-most derivation

LR parser builds right-most derivation

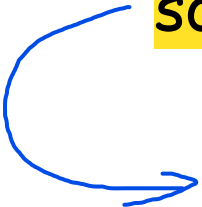
- but does so in reverse: first step of derivation is the last reduction (the reduction to start nonterminal)
- example coming in two slides

hence the name:

- L: scan input left to right
- R: right-most derivation

so, if there is a parse tree, LR parser will build it!

- this is the key theorem



Se la stringa di input appartiene al linguaggio definito dalla grammatica (cioè, se esiste un albero di parsing per quella stringa). Allora, l'algoritmo LR è garantito per trovare la corretta sequenza di Shift/Reduce che corrisponde all'inverso della derivazione più a destra, e quindi accettare l'input.

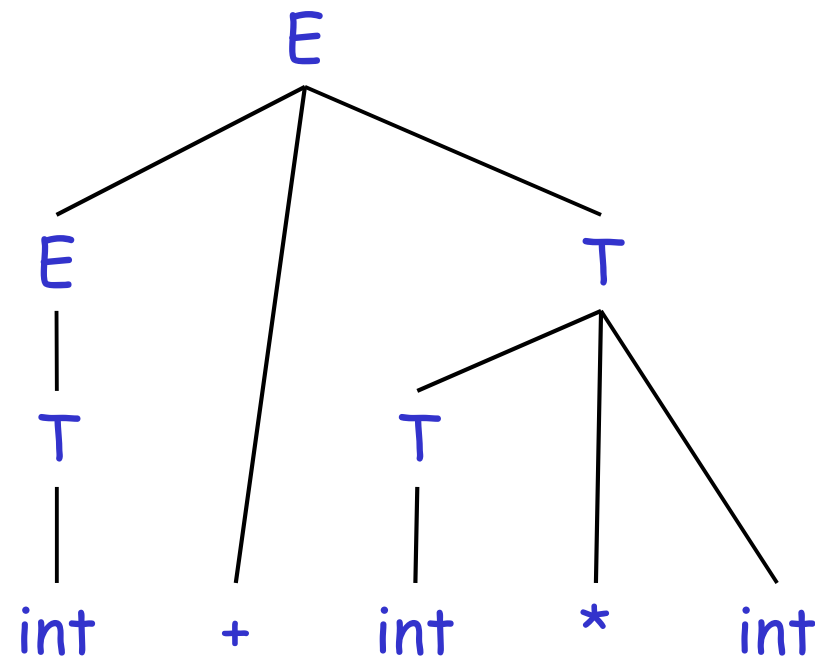
LR parser actions

- The left part of the string will be on the stack
 - the ► symbol is the top of stack
- Two simple actions
 - reduce:
 - like in chaotic parser,
 - but must replace a string on top of stack
 - shift:
 - shifts ► to the right,
 - which moves a new token from input onto stack, potentially enabling different reductions
- These actions will be chosen non-deterministically

Example of a correct LR parser sequence

A “lucky” sequence of shift/reduce actions (string parsed!):

E ▶
E + T ▶
E + T * int ▶
E + T * ▶ int
E + T ▶ * int
E + int ▶ * int
E + ▶ int * int
E ▶ + int * int
T ▶ + int * int
int ▶ + int * int
▶ int + int * int



Example of an incorrect LR parser sequence

will be stuck after reduction to $T + E$!

why can't we reduce to $E + T$, instead?

$T + T \triangleright$

$T + T * \text{int} \triangleright$

$T + T * \triangleright \text{int}$

$T + T \triangleright * \text{int}$

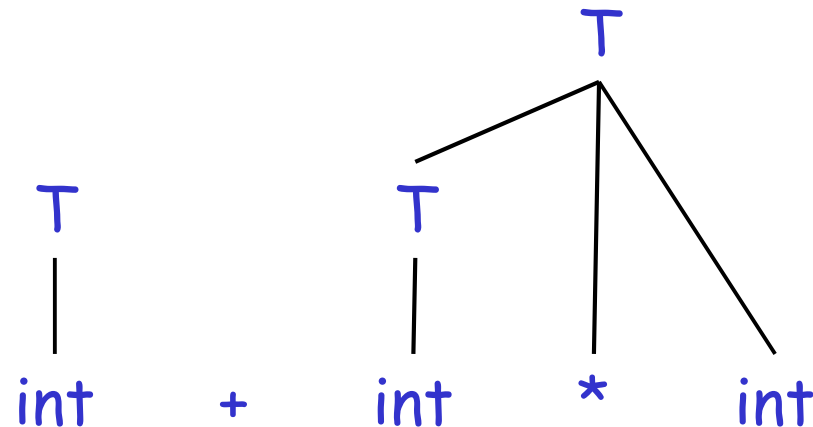
$T + \text{int} \triangleright * \text{int}$

$T + \triangleright \text{int} * \text{int}$

$T \triangleright + \text{int} * \text{int}$

$\text{int} \triangleright + \text{int} * \text{int}$

$\triangleright \text{int} + \text{int} * \text{int}$



Where did the parser instance make the mistake?

C'è sempre una derivazione corretta (a meno che ci siano degli errori sintattici) ed è una derivazione canonica a destra (il non determinismo permette di avere al massimo un successo in una istanza del parser e le altre istanze diventano bloccanti/fallite)

③

Non-deterministic LR parser

The algorithm: (compare with chaotic n.d. parser)

1. find all reductions allowed on top of stack
 - assume there are k of them
2. create k new identical instances of the parser
3. in each instance, perform one of the k reductions; in original instance, do no reduction, shift instead
 - and go to step 1
4. stop when a parser instance reduced the string to start non-terminal

Il parser esamina la cima del suo stack. Se i simboli in cima allo stack corrispondono al corpo di k diverse regole di produzione della grammatica, quelle k riduzioni sono possibili in quel momento.

Per gestire le scelte multiple, il parser si clona. Vengono create k nuove copie del parser, ognuna con lo stesso contenuto dello stack e lo stesso puntatore all'input.

(In ogni istanza, esegue una delle k riduzioni; nell'istanza originale, non esegue riduzioni, esegue invece uno shift)

- Istanze 1 a k (Reduce): Ogni nuova istanza esegue una delle k possibili riduzioni.
- Istanza Originale (Shift): L'istanza da cui sono partite le clonazioni non esegue riduzioni. Tenta invece l'operazione di Shift (sposta il prossimo simbolo di input sullo stack).
- Tutte le istanze (le k nuove ridotte e l'originale shiftata) ripetono l'intero processo tornando al Passo 1.
- Il parsing ha successo quando una singola istanza è riuscita a eseguire tutte le riduzioni necessarie, fino a che l'intero input è rappresentato dal solo simbolo iniziale sullo stack e l'input è esaurito.

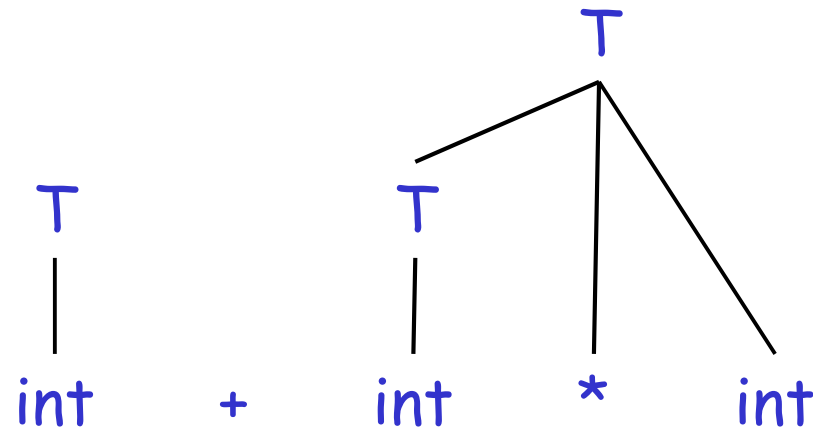
Overview

- ① • Chaotic bottom-up parser
 - tries one derivation (in reverse)
 - ② • Non-deterministic chaotic parser
 - tries all ways to build the parse tree
 - ③ • Non-deterministic LR parser
 - restricts where a reduction can be made
 - as a result,
 - only one instance succeeds (on an unambiguous grammar)
 - all others get stuck
 - ④ • Generalized LR parser (next)
 - idea: kill off instances that are going to get stuck ASAP
- ↪ il prima possibile

Obiettivo: Eliminare il Non-Determinismo perché inefficiente
Come? Terminare il prima possibile la copia del Parser che si bloccherà sicuramente andando avanti (trovare la configurazione stack condannata che dopo di essa porterà ad un insuccesso)

Revisit the incorrect LR parser sequence

T + T ▶
T + T * int ▶
T + T * ▶ int
T + T ▶ * int
T + int ▶ * int
T + ▶ int * int
T ▶ + int * int
int ▶ + int * int
▶ int + int * int



Key question:

What was the earliest stack configuration where we could tell this instance was ^{condannata} doomed to get stuck?

Doomed stack configurations

The parser made a mistake to shift to

T + ► int * int

rather than reducing to

E ► + int * int

→ The first configuration is doomed

- because the T will never appear on top of stack so
that it can be reduced to E
- hence this instance of the parser can be killed (it
will never produce a parse tree)

così che possa essere ridotta
ad E (non accadrà mai)

In generale, le configurazioni sono infinite: ciò non significa che non si possono rappresentare (dipende dalla struttura d'infinito)

Un'istanza di parser è considerata condannata se, a partire dalla configurazione attuale, è impossibile arrivare a una derivazione valida che consumi tutto il resto dell'input e termini con un albero di parsing completo.

Come facciamo a sapere se la sequenza di simboli che abbiamo sullo stack ha senso o se ci sta portando in un vicolo cieco?

How to find doomed parser instances?

- Look at their stack!
- How to tell if a stack is doomed:
 - list all ^{valid} legal (non yet doomed) stack configurations
 - if a stack is not legal, kill the instance

Listing legal stack configurations

- list prefixes (stack content) of all right-most derivations

Una configurazione è doomed se non è un prefisso di nessuna "Right-most derivation"

- describe them as a DFA

L'insieme di tutti i prefissi validi di una CFG è un Linguaggio Regolare.

- if the stack configuration is not from the DFA, it's doomed

Come listare tutte le configurazioni legali dello stack?

Our example grammar

- we'll build a parser for this grammar:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * \text{int} \mid \text{int}$$

Le transizioni mancanti nel DFA, vanno nello stato pozzo, che non é di accettazione.

- which are **prefixes** (stack content) of strings raggiungibili reached during **right-most derivations**?

- E.g. those of length one.

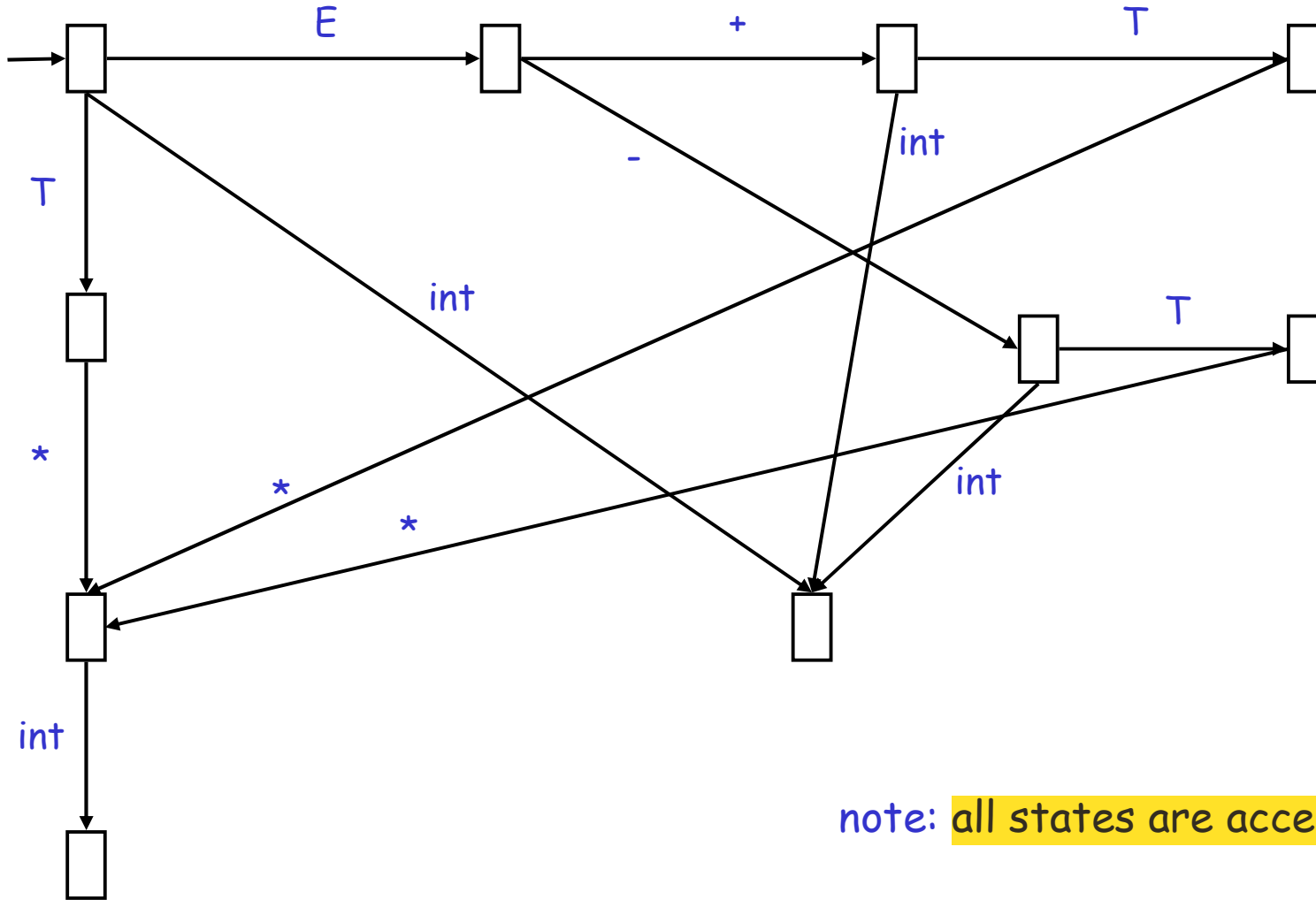
- They are "E", "T" and "int"

- Then think about those of length two, etc...

Tutti gli stati rappresentati sono stati di accettazione (stack di lunghezza 1 é legale)

The stack-checking DFA

Deve descrivere il contenuto dello stack (variabili e terminali)



note: all states are accepting states

Ci sono diverse versioni di parser (Simple LR, LR, LALR Parsing)
perché ognuna è restrittiva diversamente sulla grammatica accettata

Simple LR parsing

Viene aggiunta una nuova variabile con una produzione che punta alla vecchia variabile iniziale per capire quando il parser accetta

Introducing a new grammar initial variable

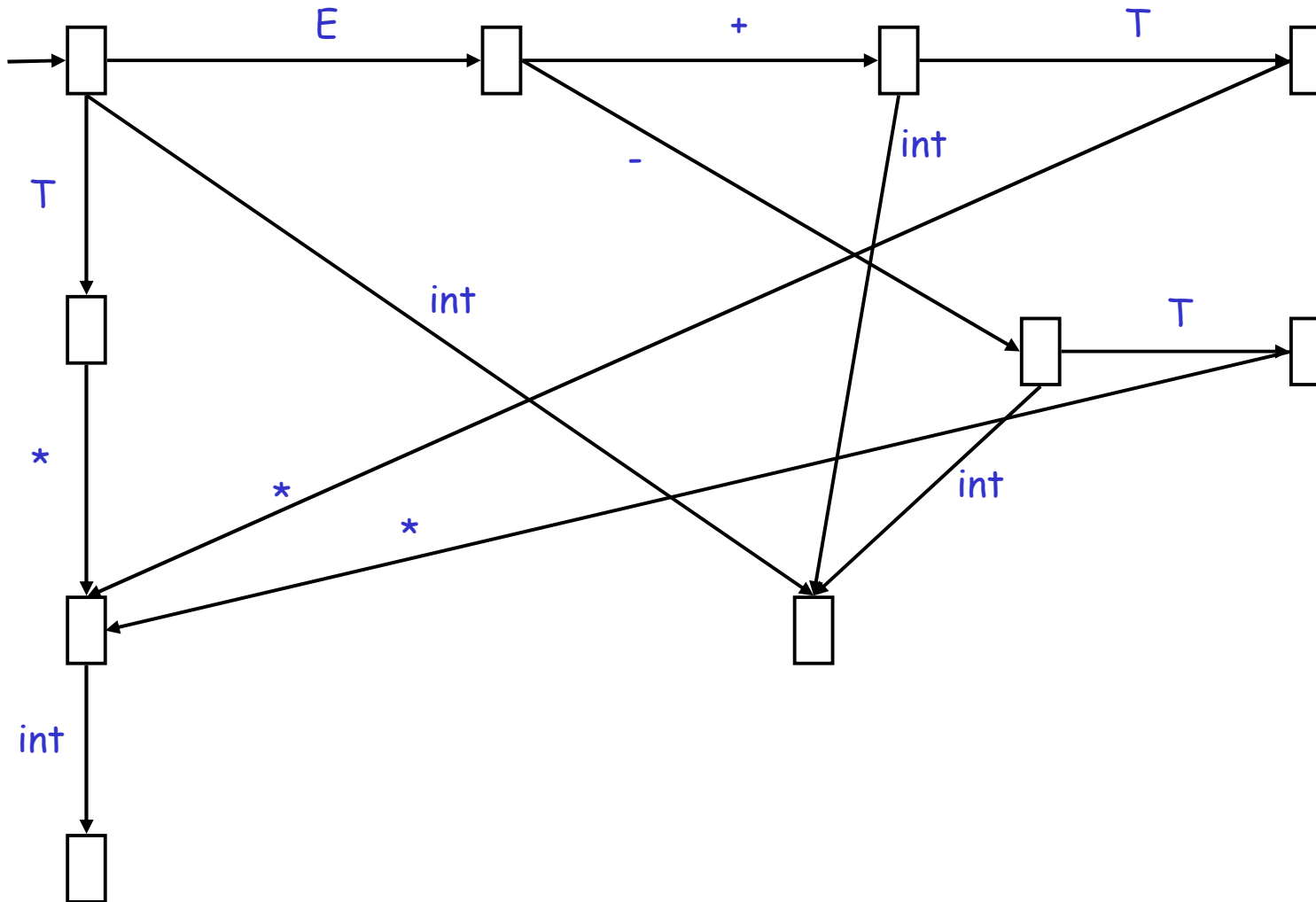
- In order to compute the stack checking DFA we preliminarily add to the grammar
 - a new initial variable E' and
 - a new production $E' \rightarrow E$
(with E being the old initial variable)
- Our example grammar thus becomes:

$$\underline{\underline{E' \rightarrow E}}$$

$$\underline{E \rightarrow E + T \mid E - T \mid T}$$

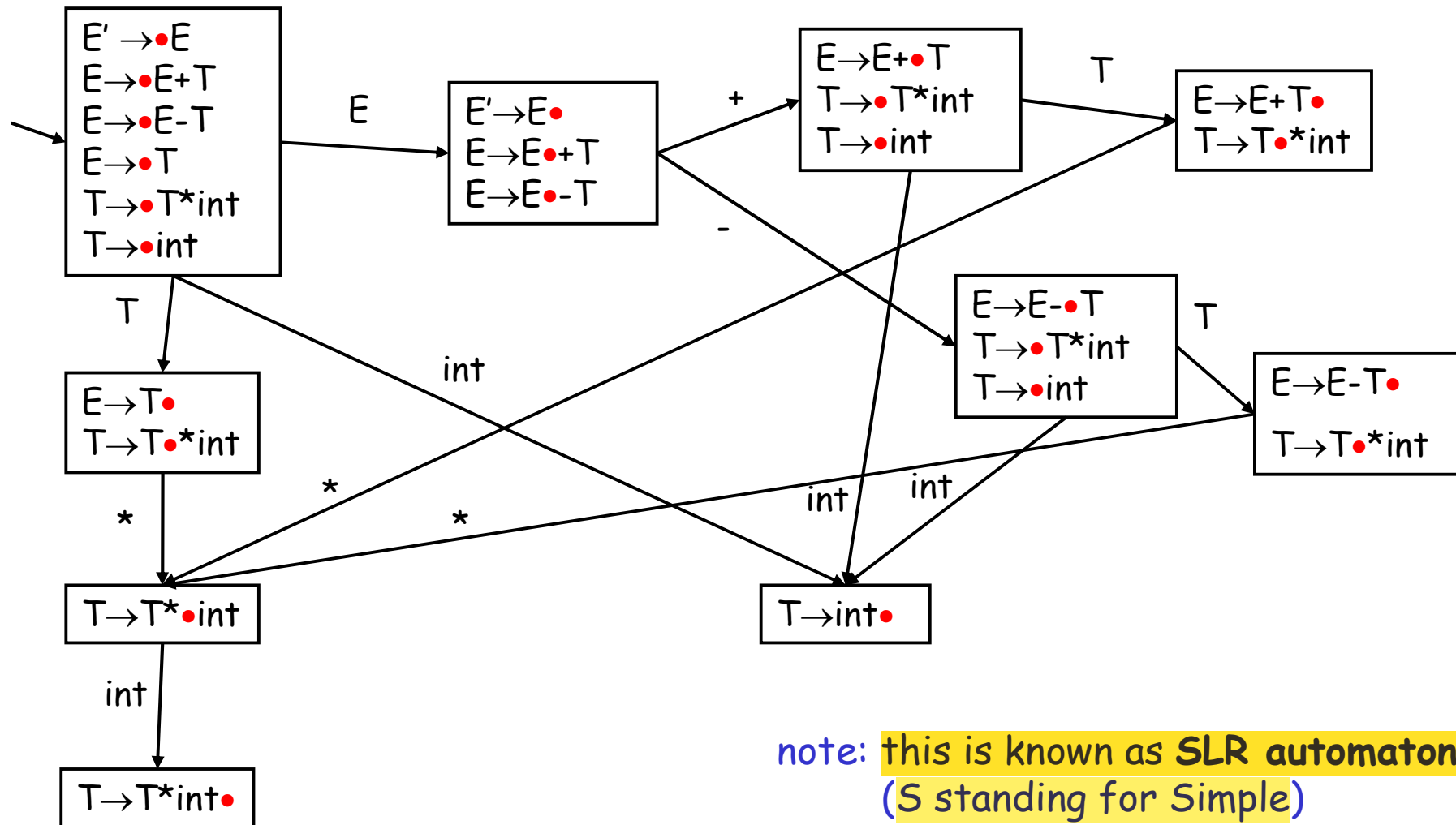
$$\underline{T \rightarrow T * \text{int} \mid \text{int}}$$

The stack-checking DFA



SLR Automaton: Ogni stato ha un insieme di produzioni (pallino rosso in qualsiasi punto del corpo). DFA utilizzato per controllo del contenuto dello stack.

Constructing the stack-checking DFA



(Produzione Pallinata)

SLR Items

Il punto indica la posizione corrente di scansione nel processo di riconoscimento di quella specifica produzione.

- A SLR item has the form:

$$X \rightarrow \alpha \bullet \beta$$

with $X \rightarrow \alpha\beta$ being a production

alpha e beta sono stringhe di simboli terminali e/o non-terminali.

$X \rightarrow \alpha \bullet \beta$ describes context information

- We are trying to find an X, and
- We have α already on top of the stack
- Thus we need to see next a string derived from β

In questo momento, in quello stato, il parser ha nello stack solo la parte a sinistra del pallino rosso ed il parser ha intenzione di mettere beta sullo stack per ridurre a X (beta può avere variabili e terminali)

Se beta è una sequenza di soli terminali, allora la "stringa derivata da beta" è beta stessa. Ci aspettiamo di vedere quei terminali nel prossimo input.
Se beta contiene non-terminali, allora il parser dovrà trovare una sequenza di terminali (token) che possa essere generata espandendo le prossime variabili.

Lo stato del DFA, quindi, non solo ci dice cosa abbiamo già riconosciuto, ma ci dice anche cosa dobbiamo iniziare a cercare nell'input per progredire (tramite l'operazione di chiusura sui non-terminali che seguono il punto). Questo insieme di item definisce precisamente le azioni Shift o Reduce che il parser può compiere in quello specifico momento e con quel preciso contenuto sullo stack.

In questo momento, il parser sta cercando di riconoscere contemporaneamente la produzione X1 (avendo già visto α_1), OPPURE la produzione X2 (avendo già visto α_2), e così via.

Ogni stato del DFA rappresenta una possibile configurazione dello stack in un dato momento del parsing. In altre parole, uno stato codifica tutte le produzioni potenziali che potrebbero essere in fase di riconoscimento a partire dai simboli che sono attualmente sullo stack.

DFA state: a set of SLR items

- A DFA state describes a parsing context: a set of SLR items representing possible productions to apply

Ciò che il parser sta cercando di fare: il parser, contemporaneamente, sta cercando di ridurre una delle produzioni nello stato attuale ed i SLR Items mostrano la produzione finale a cui posso ridurre alla testa e lo 'stato di avanzamento'

We are trying to find $E \rightarrow E + \bullet T$

- on the top of the stack we have $E +$ already

Quindi

Thus we are also trying to find $T \rightarrow \bullet T * \text{int}$ or

$T \rightarrow \bullet \text{int}$

- on the top of the stack we do not have anything yet

Contesto dello Stack: Questo è l'item che ha causato l'ingresso in questo stato. Indica che, sullo stack, abbiamo già riconosciuto la sequenza $E +$. Obiettivo Immediato: Siamo a metà della produzione per E . L'obiettivo successivo è trovare e riconoscere una stringa derivata da T .

Questi item sono la "spiegazione" di come si può riconoscere il T che ci manca nell'item principale.

Contesto Implicito: Poiché ci aspettiamo di vedere T (a causa di $E \rightarrow E + \text{punto } T$), dobbiamo considerare tutte le produzioni che definiscono T . In questi item, il punto è all'inizio, il che significa: non abbiamo ancora nulla sullo stack che corrisponda a T . Azione Implicita: Il parser deve ora iniziare a riconoscere T . Può farlo aspettandosi un terminale (int) o aspettandosi di iniziare una derivazione ricorsiva ($T * \text{int}$).

Note

In sintesi, in entrambi i casi, l'elemento a sinistra (lo stack o il prefisso alpha) è ciò che è stato riconosciuto e posizionato. L'elemento a destra (gamma o beta) è ciò che manca e deve essere letto o derivato in futuro.

Questa notazione descrive l'ambiente operativo del parser (cosa è stato visto e cosa resta da vedere nell'input).

- The symbol $\triangleright$ was used before to separate the stack from the rest of input

- $\alpha \triangleright \gamma$, where α is the stack and γ is the remaining string of terminals che devono ancora essere letti dal flusso di token in input.

Questa notazione descrive lo stato interno di una singola produzione che si sta cercando di completare.

- In SLR items $\circ$ is used to mark a prefix of a production rhs:
Right-Hand Side

$$X \rightarrow \alpha \bullet \beta$$

beta (Ancora da Riconoscere): È la stringa di simboli (che possono includere terminali e/o variabili) che si prevede di trovare o derivare in seguito.

- Here β might contain non-terminals as well
- In both cases the stack is on the left

Constructing the stack-checking DFA

- A DFA state is a ^{insieme chiuso} *closed* set of SLR(1) items
 - This means that we performed Closure
- The start state is $\text{Closure}(\{E' \rightarrow \bullet E\})$

Un insieme chiuso di SLR items si ottiene applicando l'operazione di Chiusura (Closure) ad un insieme iniziale di item.

The Closure Operation

- The operation of extending the context with items is called the **closure operation**

Closure(Items) =

repeat

for each $X \rightarrow \alpha \bullet Y \beta$ in Items

for each production $Y \rightarrow \gamma$

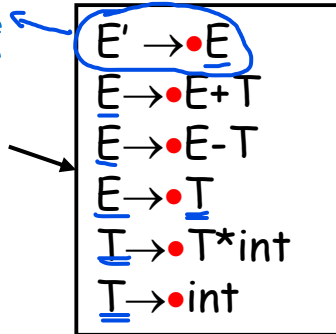
add $Y \rightarrow \bullet \gamma$ to Items

until Items is unchanged

Per ogni produzione, presente nello stato, che ha una variabile puntata dal pallino, vengono aggiunte tutte le sue produzioni in cui è testa

Constructing the stack-checking DFA

Produzione
presente prima
dell'espansione
tramite Closure



The DFA Transitions

- A state $State$ that contains at least an item $X \rightarrow \alpha \bullet y \beta$ has a transition labeled y to a state that contains the items: $Transition(State, y)$
 - y can be a terminal or a non-terminal

$Transition(State, y) =$

$Items \leftarrow \emptyset$

for each $X \rightarrow \alpha \bullet y \beta \in State$

add $X \rightarrow \alpha y \bullet \beta$ to $Items$

return $Closure(Items)$

L'operazione genera il nuovo stato (l'insieme di item) verso cui ci si muove dopo aver consumato il simbolo y .
Questo processo $[Transition(State, y)]$ si scompone in due passaggi:

- Passo 1: Spostare il Punto:

> Si inizia con un insieme vuoto di item, qui chiamato $Items$.

> Per ogni item nello $State$ corrente che ha il simbolo y subito dopo il punto ($X \rightarrow \alpha \bullet y \beta$), si crea un nuovo item semplicemente spostando il punto oltre y .

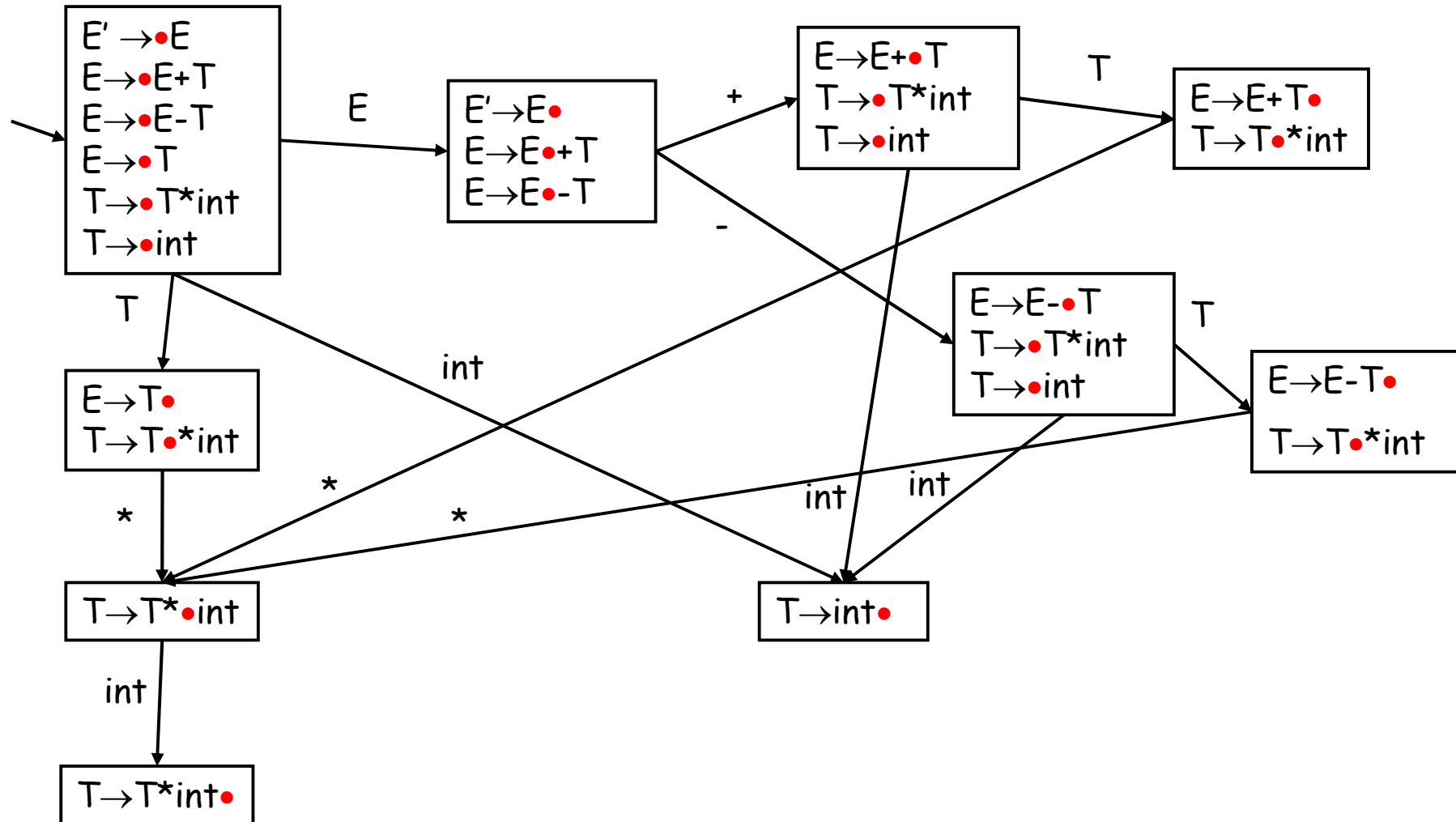
> Si aggiunge ($X \rightarrow \alpha y \bullet \beta$) ad $Items$

Questi item, dove il punto è stato spostato, formano il nucleo (core) del nuovo stato. Rappresentano l'informazione diretta su ciò che è appena stato riconosciuto.

- Passo 2: Applicare la Chiusura (Closure):

> Si applica l'operazione di Chiusura all'insieme di item appena generato ($Items$). Motivazione: L'operazione di Closure aggiunge tutti gli item derivati. Se il nuovo item principale ha il punto che precede un altro non-terminale Z , la chiusura garantisce che lo stato includa subito tutti i modi in cui Z può iniziare. Questo mantiene lo stato semanticamente completo.

Constructing the stack-checking DFA

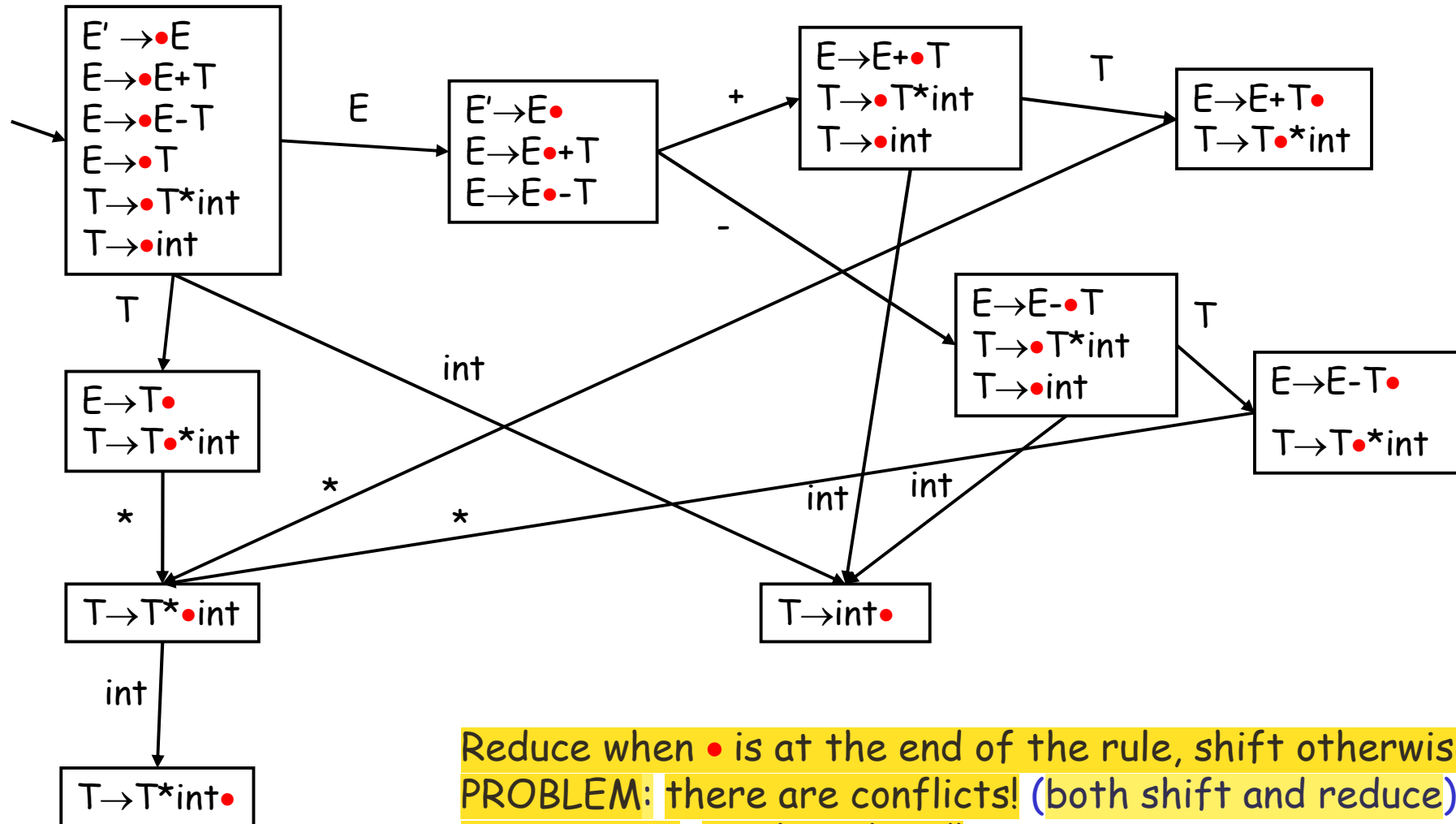


How to use the SLR automaton

Quindi, mi permette di definire le configurazioni stack corrette e mi permette di definire quando applicare reduce e shift

-
- The automaton indicates the correct stack configurations...
 - ...but it also dictates when to do shift/reduce actions...

Shift/reduce based on stack-checking DFA



Reduce when $\bullet$ is at the end of the rule, shift otherwise

PROBLEM: there are conflicts! (both shift and reduce)

SOLUTION: use lookahead!

perché ho la possibilità di fare sia Reduce sia Shift se ho il pallino alla fine in un SLR item (reduce) e transizione dallo stato (shift)

Example of a correct LR parser sequence

Prossimo Obiettivo: rimuovere non-determinismo

If we do not use lookahead ("*") we still have non-determinism!

The lookahead, instead, tells that here we cannot reduce $E+T$ to E

- because it would cause E to be followed by "*" and we know $* \notin \text{Follow}(E)$

(Senza uso di lookahead) Questo è un conflitto Shift/Reduce, ovvero Non-Determinismo. Senza ulteriori informazioni, il parser non sa cosa fare.

$E \triangleright$

$E + T \triangleright$

$E + T * \text{int} \triangleright$

$E + T * \triangleright \text{int}$

$E + T \triangleright * \text{int}$

$E + \text{int} \triangleright * \text{int}$

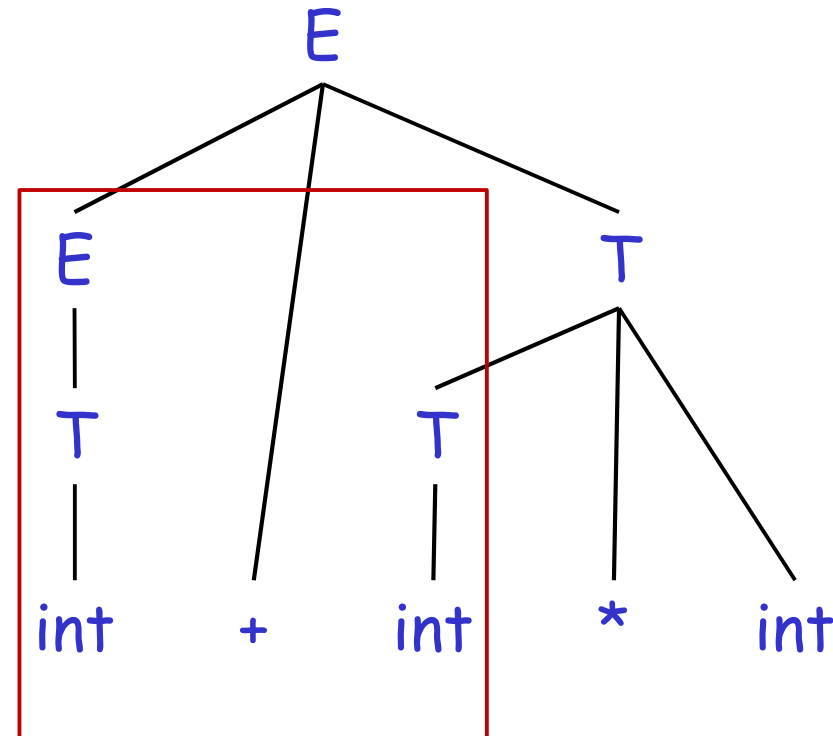
$E + \triangleright \text{int} * \text{int}$

$E \triangleright + \text{int} * \text{int}$

$T \triangleright + \text{int} * \text{int}$

$\text{int} \triangleright + \text{int} * \text{int}$

$\triangleright \text{int} + \text{int} * \text{int}$



How to use the SLR automaton

Utilizzando First Sets e Follow Set, tramite look-ahead, posso rimuovere del tutto il non-determinismo

- **Lookahead for "shift":**

- "Shift" only on the expected terminals

L'Informazione dell'Automa: Se uno stato S_i ha un SLR item $X \rightarrow \alpha \cdot t \beta$, dove t è un terminale (simbolo di lookahead), significa che il parser si aspetta di vedere t .
La Decisione SLR: Se il simbolo di lookahead corrente è esattamente t , l'azione corretta è Shift t (scorrimento) e transizione al nuovo stato S_j .

- **Lookahead for "reduce":**

- "Reduce" only on terminals in the Follow set of the variable on the l.h.s. of the rule used to reduce

Left-Hand Side

- L'Informazione dell'Automa: Se uno stato S_i contiene un Item di Riduzione ($X \rightarrow \alpha \cdot$), l'automa è pronto per eseguire una riduzione usando la produzione $X \rightarrow \alpha$.
- La Regola SLR: La riduzione non viene eseguita incondizionatamente. Viene eseguita solo se il simbolo di lookahead corrente t appartiene all'insieme dei FOLLOW(X).
 - > X è il Non-Terminale a sinistra della regola (LHS).
 - > FOLLOW(X) è l'insieme di tutti i terminali che possono legalmente seguire X in una qualsiasi forma sentenziale valida derivabile dalla grammatica.
- Decisione:
 - > Se t appartiene al FOLLOW(X): Esegui la riduzione $X \rightarrow \alpha$. (Togli i simboli di α dallo stack, consulta il GoTo con X , spingi il nuovo stato).
 - > Altrimenti (t non appartiene al FOLLOW(X)): Non eseguire la riduzione. (la riduzione viene saltata, favorendo spesso lo Shift, o portando a un errore di sintassi).

Algoritmo per la creazione
della tabella di Parsing
(Parsing Deterministico)



- 1) Calcolo First Set e Follow Sets
- 2) Definizione SLR Automaton (gli stati diventeranno le righe della tabella di parsing, le transizioni diventeranno le colonne).
- 3) Costruzione Parsing Table (aggiungo nelle celle anche Shift, Reduce e GoTo)

Computation of first and follow sets

- Consider our example grammar

$$E' \rightarrow \underline{E}$$

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * \text{int} \mid \text{int}$$

First(E')={int} (no need to calculate it) perché First(E)=First(E')

First(E)={int}

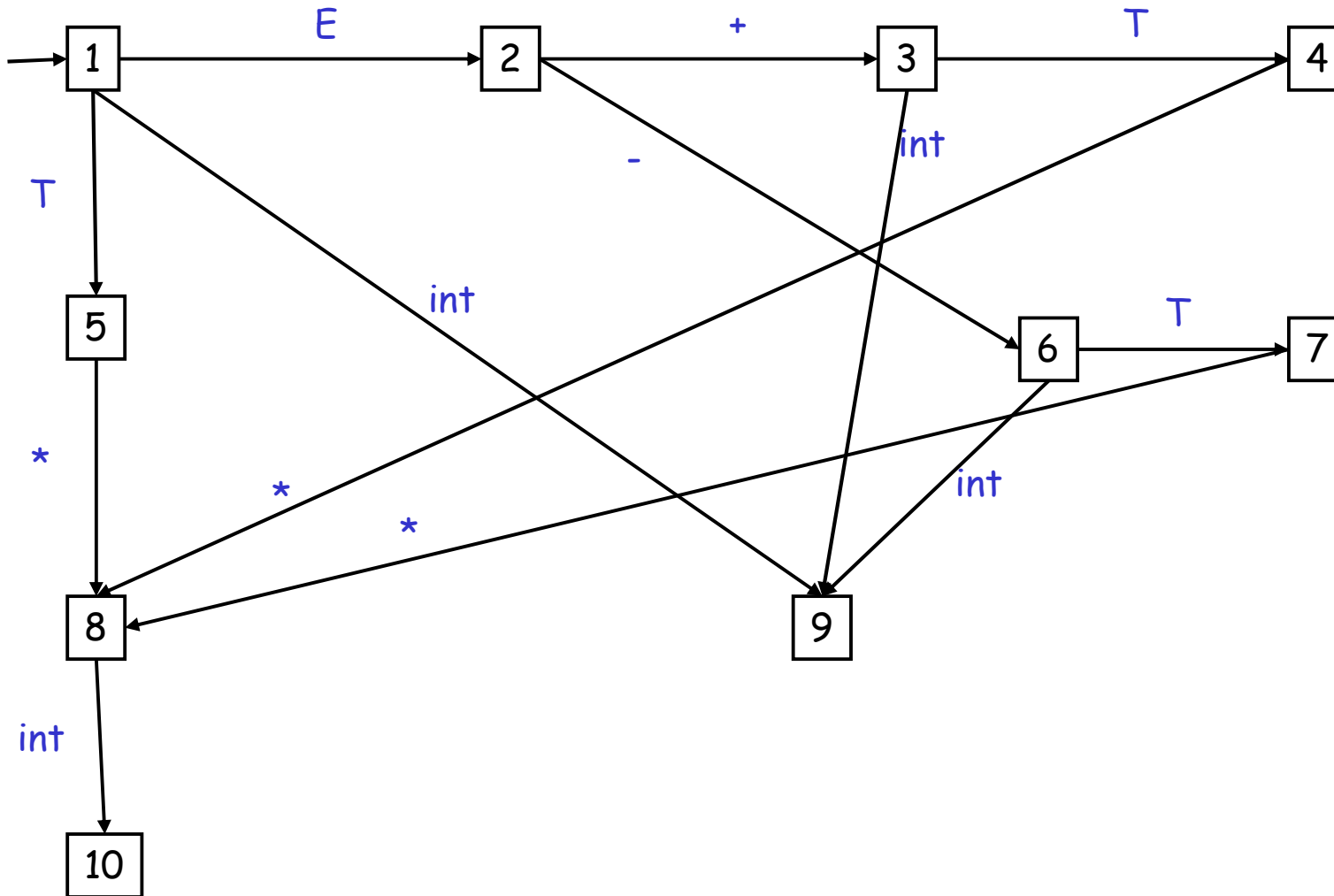
First(T)={int}

Follow(E')={\$} (no need to calculate it, always {\$})

Follow(E)={\$,+,-}

Follow(T)={\$,+,-,*}

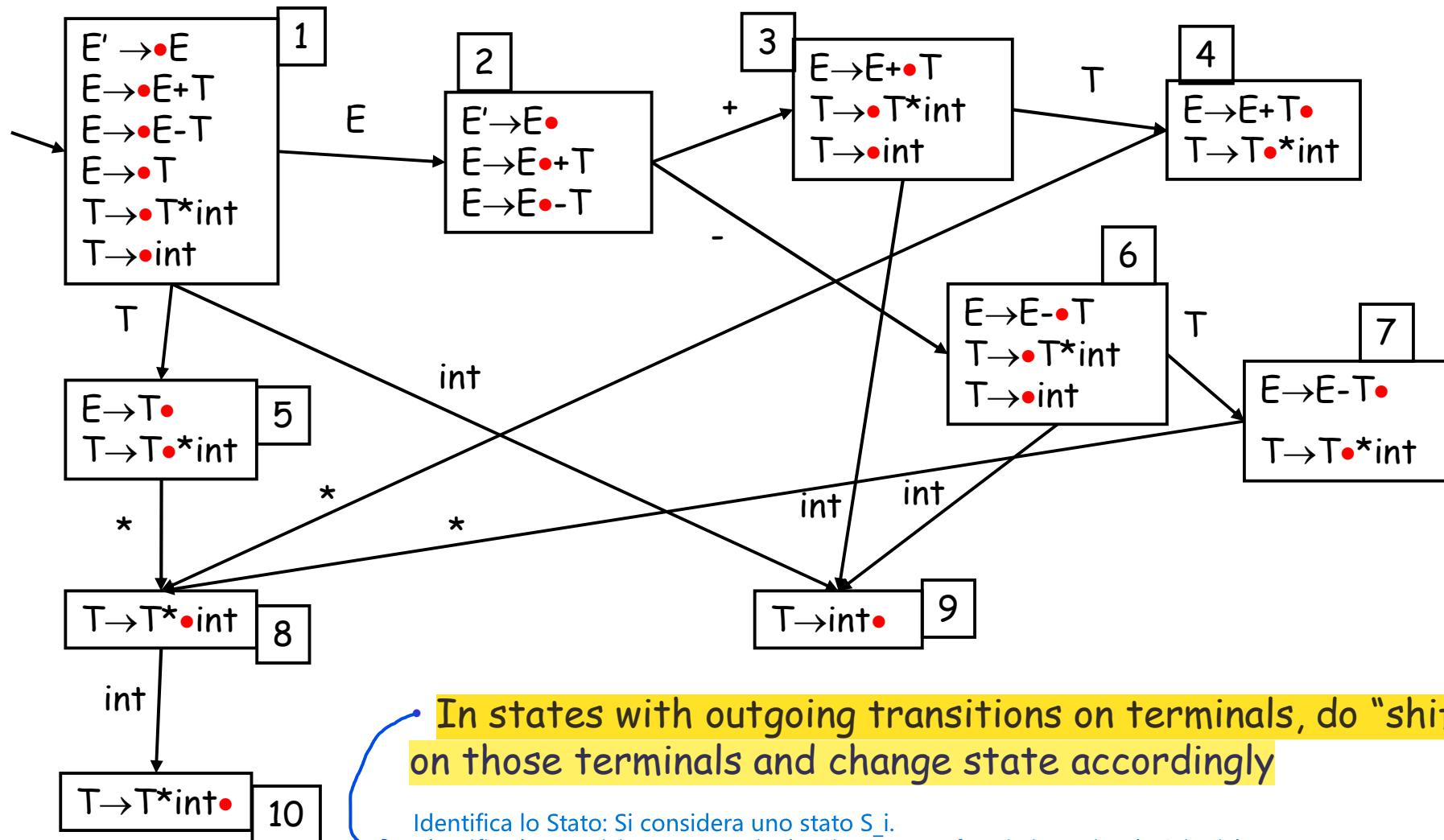
How to use the SLR automaton



How to use the SLR automaton (tabular representation)

	int	+	-	*	E	T
1	9				2	5
2		3	6			
3	9					4
4				8		
5				8		
6	9					7
7				8		
8	10					
9						
10						

How to use the SLR automaton



In states with outgoing transitions on terminals, do "shift" on those terminals and change state accordingly

Identifica lo Stato: Si considera uno stato S_i .

Identifica la Transizione su Terminale: Si cerca una freccia in uscita da S_i etichettata con un terminale t .

Identifica lo Stato Successivo: La freccia porta a un nuovo stato S_j .

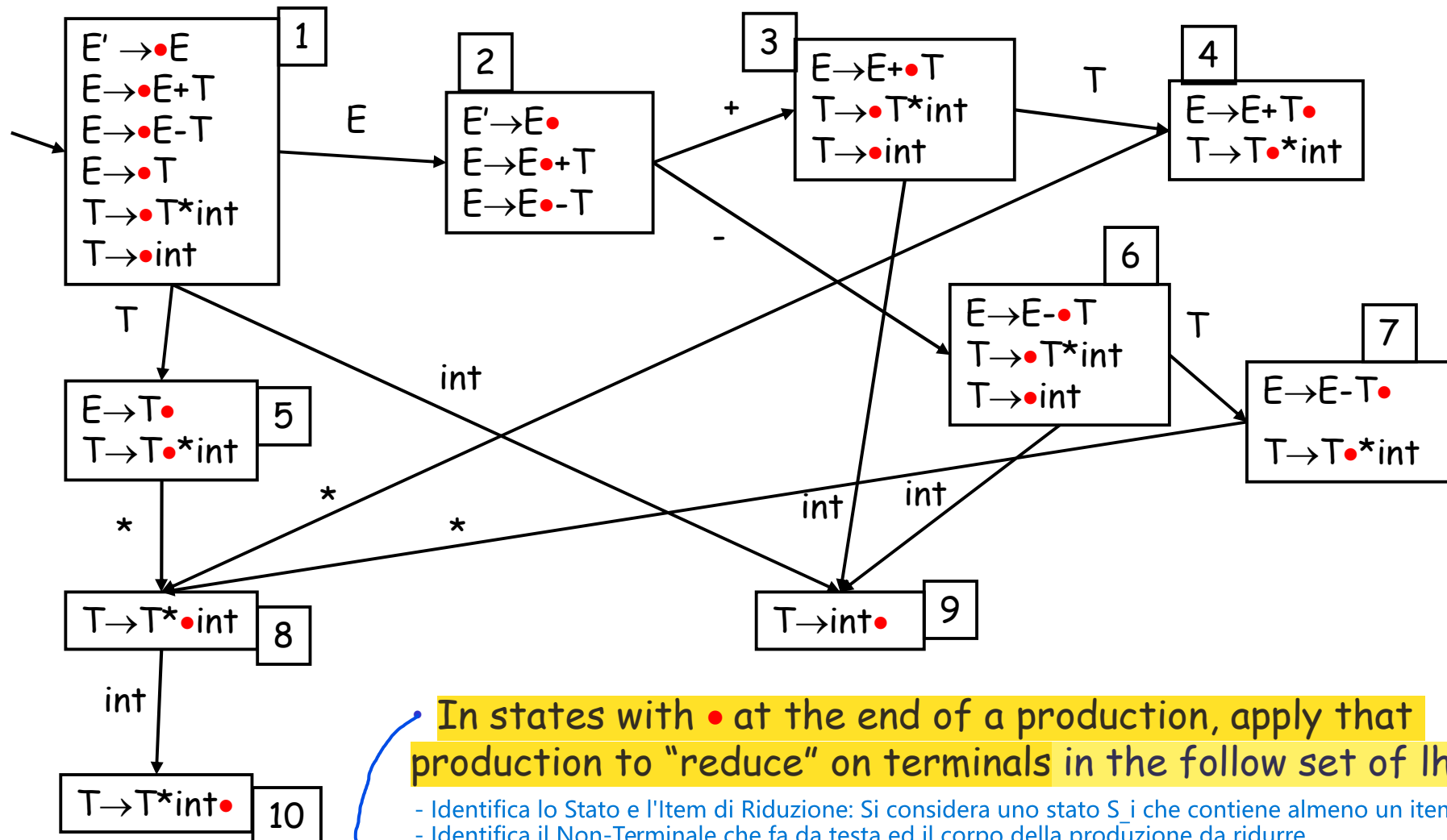
Definisci l'Azione: Nella cella ACTION della Tabella di Parsing, alla riga S_i e colonna t , si scrive l'azione Shift a S_j .

Metto nella tabella, prima gli SHIFT, da stato i a stato j (riga) con transizione un terminale (colonna) (se variabile, é un GoTo).

How to use the SLR automaton (tabular representation)

	int	+	-	*	E	T
1	shift,9				2	5
2		shift,3	shift,6			
3	shift,9					4
4				shift,8		
5				shift,8		
6	shift,9					7
7				shift,8		
8	shift,10					
9						
10						

How to use the SLR automaton



In states with $\bullet$ at the end of a production, apply that production to "reduce" on terminals in the follow set of lhs

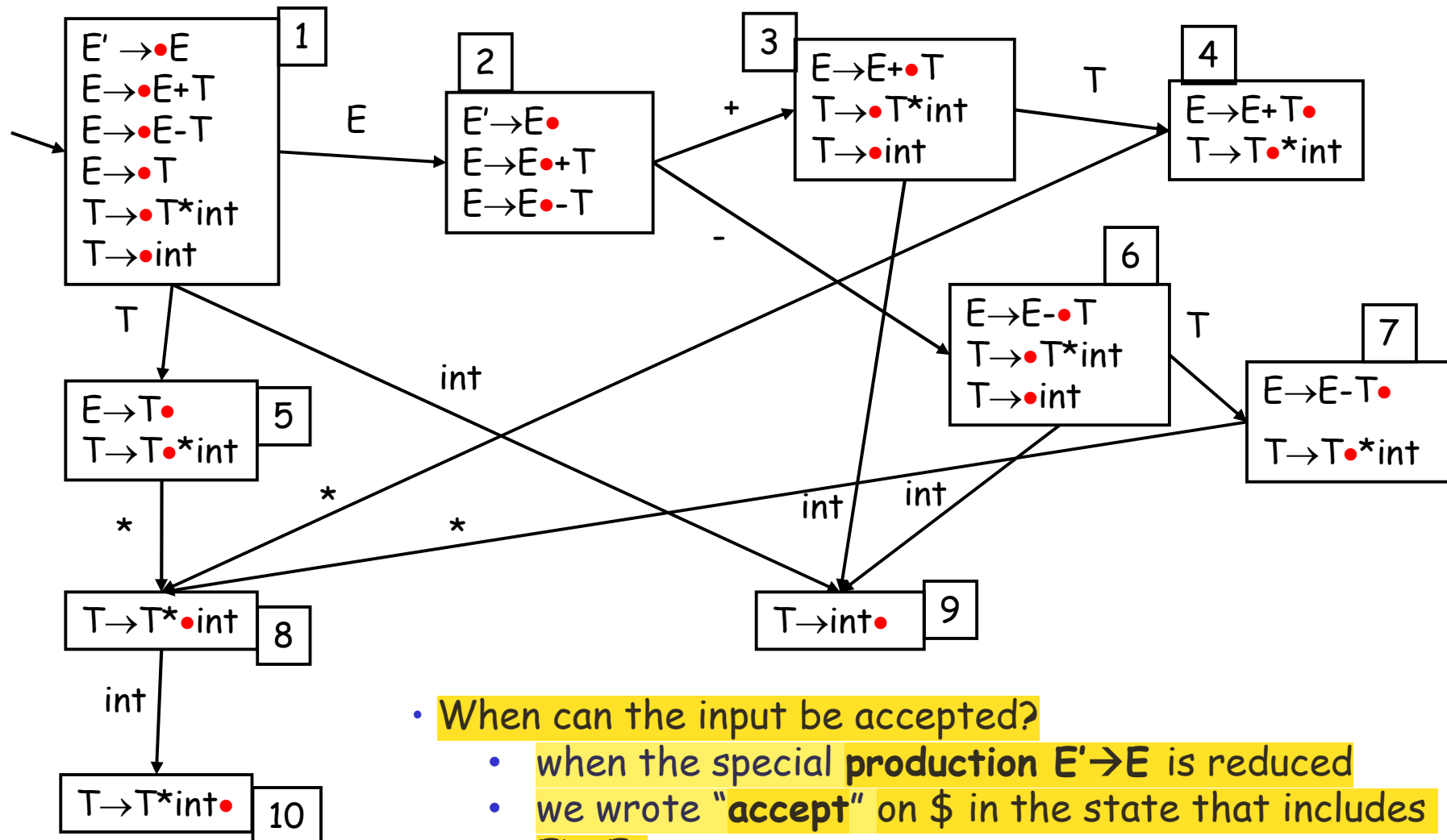
- Identifica lo Stato e l'Item di Riduzione: Si considera uno stato S_i che contiene almeno un item di riduzione.
- Identifica il Non-Terminale che fa da testa ed il corpo della produzione da ridurre.
- Calcola il FOLLOW(X): Per rendere la riduzione deterministica e corretta, si calcola l'insieme FOLLOW(X) (tutti i terminali che possono seguire X in una derivazione valida).
- Definisci l'Azione: Per ogni terminale t che appartiene all'insieme FOLLOW(X), si scrive l'azione Reduce con la produzione nella cella ACTION della Tabella di Parsing, alla riga S_i e colonna t .

Aggiunto perché la riduzione con produzione $E' \rightarrow E$, mi dice che l'automa accetta

How to use the SLR automaton (tabular representation, \$ column added)

	int	+	-	*	\$	E	T
1	shift,9					2	5
2		shift,3	shift,6		accept		
3	shift,9						4
4		red, $E \rightarrow E+T$	red, $E \rightarrow E+T$	shift,8	red, $E \rightarrow E+T$		
5		red, $E \rightarrow T$	red, $E \rightarrow T$	shift,8	red, $E \rightarrow T$		
6	shift,9						7
7		red, $E \rightarrow E-T$	red, $E \rightarrow E-T$	shift,8	red, $E \rightarrow E-T$		
8	shift,10						
9		red, $T \rightarrow \text{int}$	red, $T \rightarrow \text{int}$	red, $T \rightarrow \text{int}$	red, $T \rightarrow \text{int}$		
10		red, $T \rightarrow T*\text{int}$	red, $T \rightarrow T*\text{int}$	red, $T \rightarrow T*\text{int}$	red, $T \rightarrow T*\text{int}$		

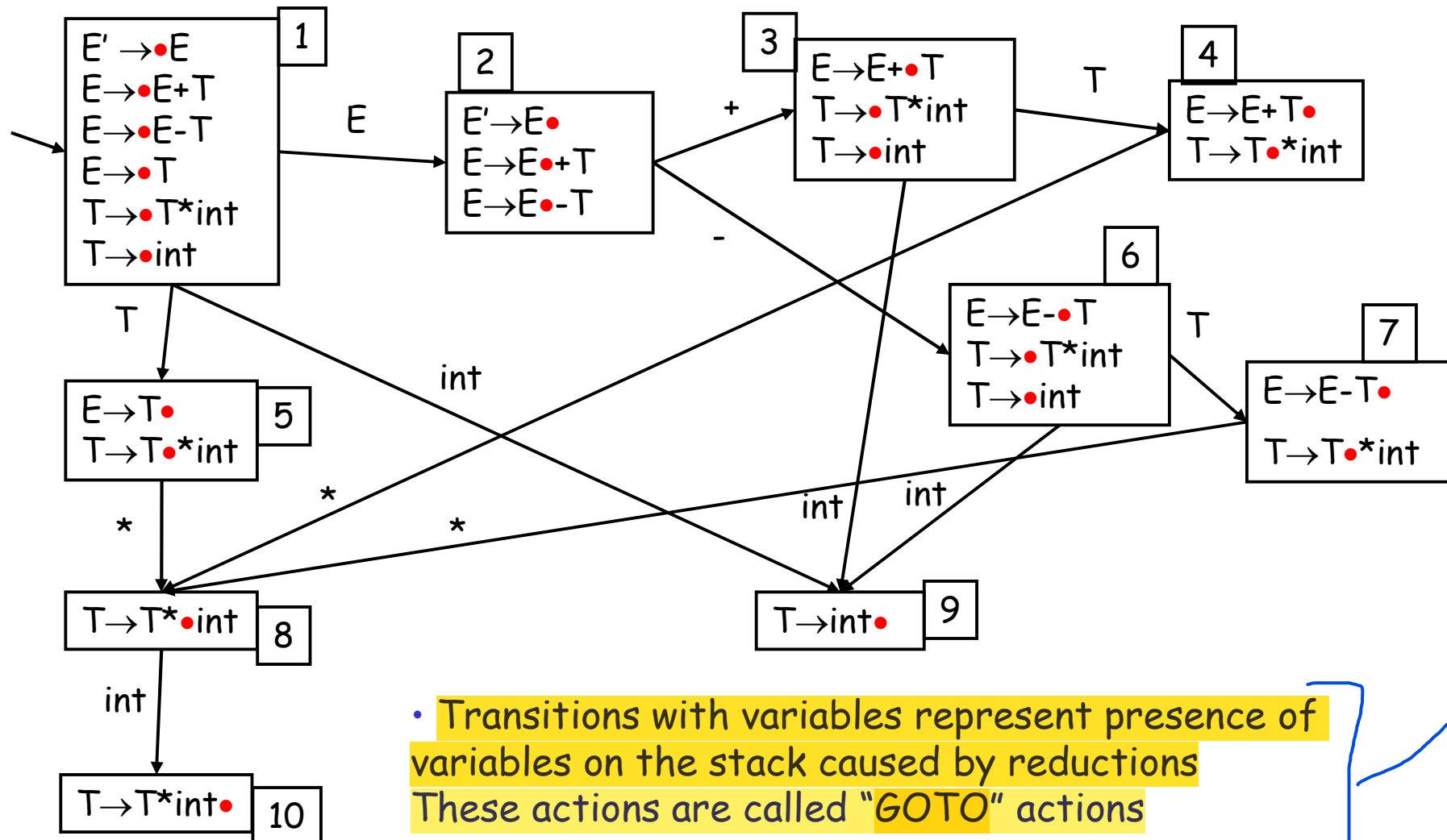
How to use the SLR automaton



- When can the input be accepted?
 - when the special production $E' \rightarrow E$ is reduced
 - we wrote "accept" on \$ in the state that includes $E' \rightarrow E \bullet$

Quando il parser esegue un'azione di riduzione con una regola $X \rightarrow \alpha$, succedono tre cose: La stringa α viene rimossa dalla cima dello stack (inclusi i numeri di stato associati). Lo stack torna ad uno stato precedente S_k . Il Non-Terminale X (la testa della produzione) viene pushato sullo stack. L'azione GoTo è la funzione che determina quale nuovo stato deve essere spinto sullo stack dopo che il Non-Terminale X è stato riconosciuto. Definizione: La funzione $GoTo(S_k, X)$ è definita dalla transizione etichettata con il Non-Terminale X dallo stato S_k . Procedura: Si recupera lo stato S_k (in cui arrivo dopo riduzione) che era sotto i simboli di α sullo stack. Si consulta la tabella di parsing per la transizione $GoTo(S_k, X)$. Lo stato di destinazione S_j viene pushato sullo stack insieme al simbolo X .

How to use the SLR automaton



How to use the SLR automaton (complete tabular representation)

Dove, dopo la riduzione, la riga é lo stato in cui mi trovo dopo la riduzione, mentre la colonna é la testa della produzione ridotta

	int	+	-	*	\$	E	T
1	shift,9					goto,2	goto,5
2		shift,3	shift,6		accept		
3	shift,9						goto,4
4		red, $E \rightarrow E+T$	red, $E \rightarrow E+T$	shift,8	red, $E \rightarrow E+T$		
5		red, $E \rightarrow T$	red, $E \rightarrow T$	shift,8	red, $E \rightarrow T$		
6	shift,9						goto,7
7		red, $E \rightarrow E-T$	red, $E \rightarrow E-T$	shift,8	red, $E \rightarrow E-T$		
8	shift,10						
9		red, $T \rightarrow \text{int}$	red, $T \rightarrow \text{int}$	red, $T \rightarrow \text{int}$	red, $T \rightarrow \text{int}$		
10		red, $T \rightarrow T*\text{int}$	red, $T \rightarrow T*\text{int}$	red, $T \rightarrow T*\text{int}$	red, $T \rightarrow T*\text{int}$		

Si chiama "predittivo" (anche se è Bottom-Up) perché, grazie al Lookahead (il simbolo successivo che "sbirciamo" nell'input), il parser può "prevedere" qual è l'unica mossa corretta da fare senza dover andare per tentativi.

- SLR = Simple LR.
- (1) = Indica che guardiamo 1 solo simbolo di lookahead per decidere se fare shift o reduce.

How to use the SLR automaton

- The LR predictive algorithm governed by this table is called **SLR(1) parsing algorithm**
 - Note that it works only if each cell of the table contains at most one action! (i.e. no conflict)
 - In this case, the grammar is a SLR(1) grammar

↓

Con alcune grammatiche, posso avere in una cella sia Shift sia Reduce (conflitto Shift-Reduce) oppure due Reduce (conflitto Reduce-Reduce). Quindi, questa grammatica, da cui ho generato la tabella, non è SLR(1), cioè non basta un solo token in look-ahead (il parser non sa se fare Shift o Reduce oppure fare la prima o la seconda Reduce).

Bottom-up parsing algorithm

Input: La stringa di terminali.
Obiettivo: Trasformare la stringa di input nel simbolo di inizio S.
Processo (Iterativo): Finché la stringa corrente (str) non è S, cerca una sottostringa beta in str che sia una Maniglia (Handle): cioè beta è il lato destro di una produzione $A \rightarrow \beta$. beta è la parte più a sinistra del corpo della regola che può essere ridotta per produrre la derivazione più a destra in senso inverso. Riduzione: Sostituisci beta con il simbolo A nella stringa str. Questa sequenza di riduzioni è esattamente la derivazione più a destra della grammatica, letta all'indietro.

Remember the idea of LR parsing

- LR parsing *reduces* a string to the start symbol by inverting productions:

str such that $str \rightarrow^* \text{input string of terminals}$

while str $\neq$ S: variabile iniziale

- Identify β in str such that $A \rightarrow \beta$ is a production and $S \rightarrow^* \alpha A \gamma \rightarrow \alpha \beta \gamma = \text{str}$
- Replace β by A in str (so $\alpha A \gamma$ becomes new str)

Stronger than top-down parsing!

- make decisions *after* seeing all symbols β rather than before (LL(1) make decisions knowing at most one of those symbols, the lookahead)

A parità di token in look-ahead, SLR é molto più potente di LL perché la decisione viene presa dopo aver visto tutto il corpo di una produzione + look-ahead (bottom-up) mentre in top-down vedo solo la testa della produzione e decido in base al look-ahead

Bottom-up parsing algorithm

- Add \$ at the end of the input
- Execute the bottom-up parsing algorithm using an automaton (e.g. the SLR automaton) to decide whether to shift or reduce
 - Every time a decision should be taken, use the stack as input for the automaton
 - The automaton will communicate whether to shift or reduce (based on the lookahead)

A Running Example

- We will study the bottom-up parsing algorithm considering

- The grammar:

$$E \rightarrow E + (E) \mid \text{int}$$

- An automaton/parsing table for such a grammar, i.e. its SLR(1) automaton

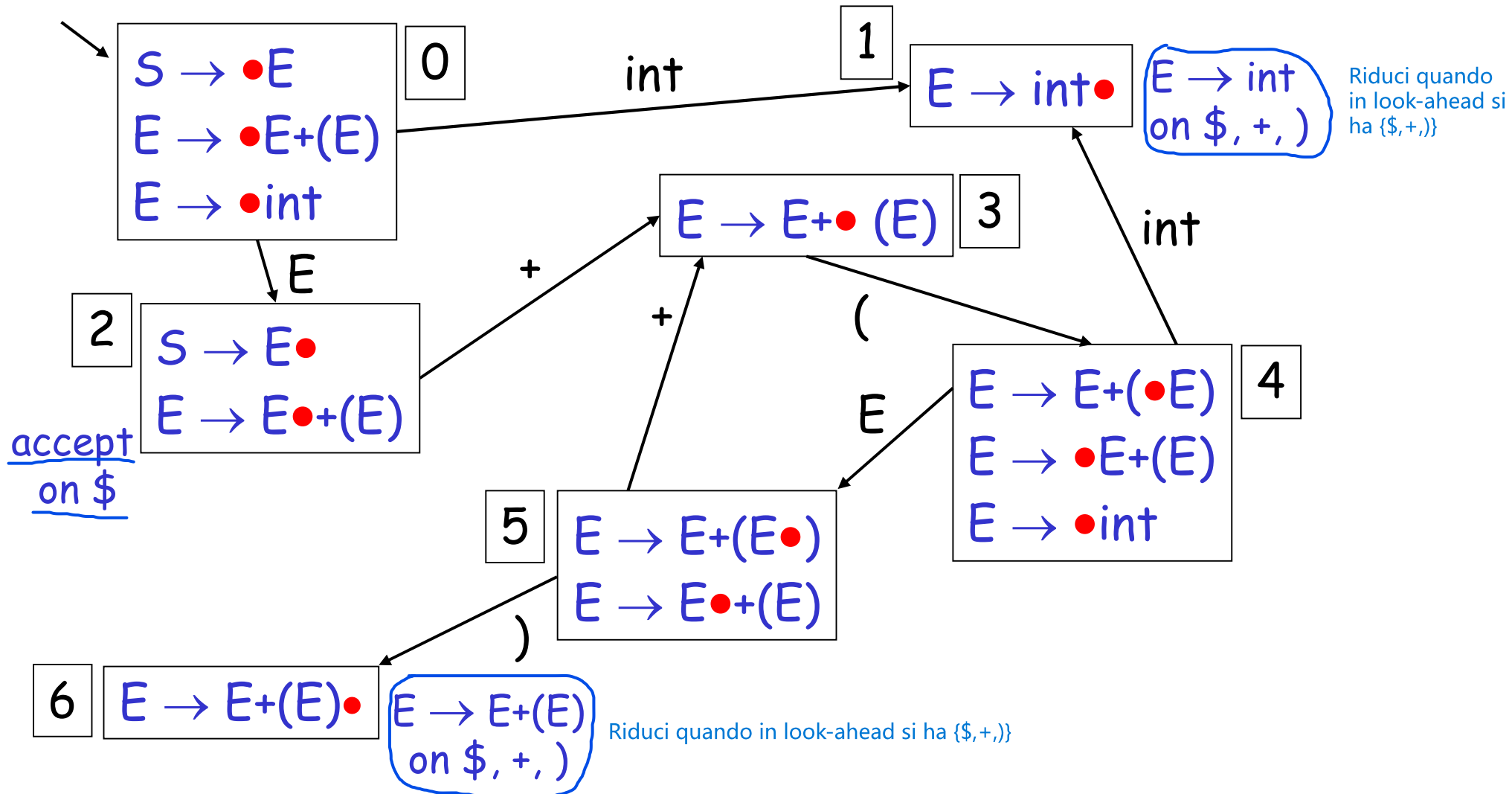
①

- $\text{First}(E) = \{\text{int}\}$
- $\text{Follow}(E) = \{\$, +,)\}$

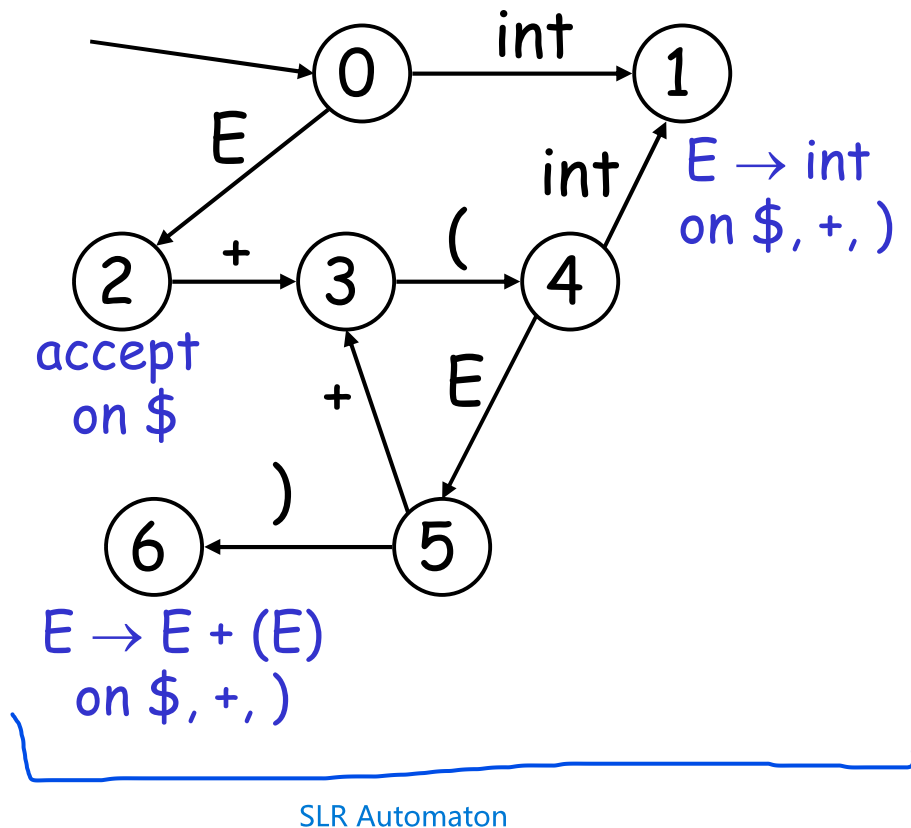
1) Calcolo First Set e Follow Sets delle variabili e le riempo con soli terminali

Quando si aggiungono nuove produzioni (grazie alle Closure),
bisogna mettere il pallino all'inizio del corpo della produzione

② Constructing the Parsing DFA. Example.



Bottom-up parsing algorithm example



int + (int) + (int)\$	shift	
<u>int</u> ▶ <u>+</u> (int) + (int)\$	<u>E → int</u>	Reduce
E ▶ + (int) + (int)\$	shift(x3)	
E + (int ▶) + (int)\$	E → int	Reduce
E + (E ▶) + (int)\$	shift	
E + (E) ▶ + (int)\$	E → E+(E)	Reduce
E ▶ + (int)\$	shift (x3)	
E + (int ▶)\$	E → int	Reduce
E + (E ▶)\$	shift	
E + (E) ▶ \$	E → E+(E)	Reduce
E ▶ \$	accept	

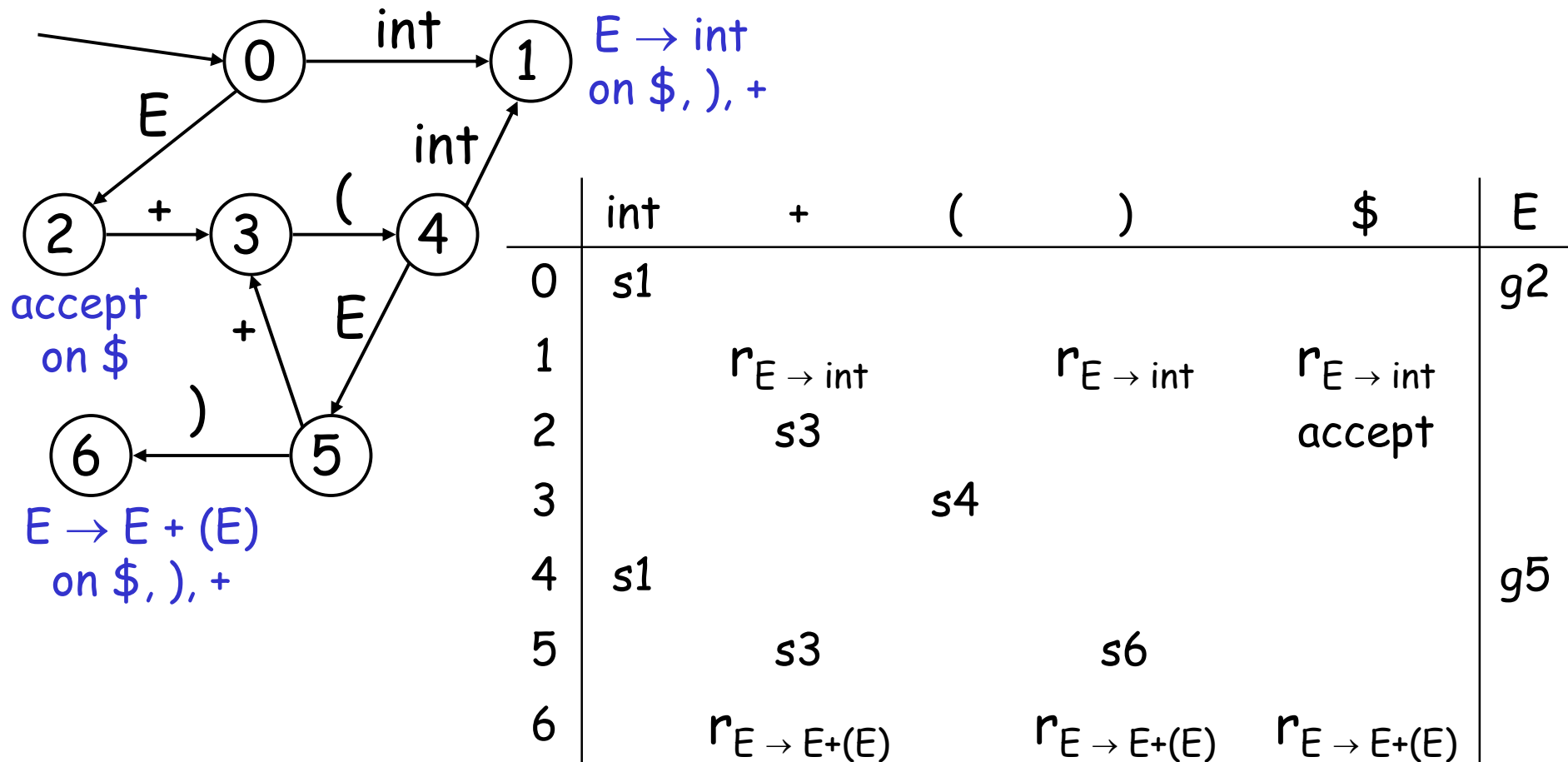
Contenuto Stack + Input + Actions

DFA table representation is actually used

- Parsers represent the DFA as a 2D table
 - As for table-driven lexical analysis
- Lines correspond to DFA states
- Columns correspond to terminals and non-terminals
- In classical treatments, columns are split into:
 - Those for terminals: action table
 - Those for non-terminals: goto table

Representing the DFA. Example

- The table for our DFA:



Se, dopo ogni azione (Shift o Reduce), il parser dovesse ripartire dallo stato iniziale (Stato 1) e ripercorrere l'intera sequenza di simboli attualmente presenti sullo stack per determinare lo stato corrente, il processo sarebbe estremamente inefficiente ($O(n^2)$ o peggio).

Grazie ai GoTo ed alle coppie (simbolo, stato) posso rendere l'algoritmo ancora più efficiente (non torno sempre all'inizio dell'automa, quindi non devo stare a seguire il percorso dell'automa). Ciò mi permette di utilizzare solo la parsing table, stack e stringa in input

The Bottom-up Parsing Algorithm

- After a shift or reduce action we rerun the DFA on the entire stack
 - This is wasteful, since most of the work is repeated
- So record, for each stack element, state of the DFA after that element

- Parser maintains a stack

$\langle \text{sym}_1, \text{state}_1 \rangle \dots \langle \text{sym}_n, \text{state}_n \rangle$

state_k is the final state of the DFA on $\text{sym}_1 \dots \text{sym}_k$

La soluzione è trasformare lo stack in una sequenza di coppie (simbolo, stato). Non solo si memorizza il simbolo di grammatica riconosciuto (sym_k), ma anche lo stato del DFA che si è raggiunto dopo aver processato tutti i simboli fino a quel punto (state_k).

Efficienza Lineare ($O(n)$): Dopo uno Shift o una Reduce, il DFA viene "eseguito" solo per l'ultimo simbolo aggiunto. L'operazione è costante ($O(1)$) per simbolo, rendendo il parsing efficiente.

Azione Shift: Se l'azione è Shift del terminale t allo stato S_{new} : Si spinge la coppia $\langle t, S_{\text{new}} \rangle$ sullo stack. Il nuovo stato corrente è S_{new} .

Azione Reduce: Se l'azione è Reduce, si rimuovono dallo stack $|\alpha|$ coppie $\langle \text{sym}, \text{state} \rangle$. Si osserva lo stato S_{old} che si trova ora in cima (il secondo elemento della nuova cima). Si calcola il $\text{GoTo}(S_{\text{old}}, X) = S_{\text{new}}$. Si pusha la nuova coppia $\langle X, S_{\text{new}} \rangle$ sullo stack.

The Bottom-up Parsing Algorithm

Let $I = w_1 w_2 \dots w_n \$$ be initial input

Let $j = 1$

Let DFA state 0 be the start state

Let $\text{stack} = \langle \text{dummy}, 0 \rangle$

repeat

È lo stato S_i che si trova in cima allo stack
(il secondo elemento della coppia in cima).

case action[$\text{top_state}(\text{stack}), I[j]$] of

shift k : push $\langle I[j], k \rangle$; $j += 1$

look-ahead

Si consuma il lookahead $I[j]$. Si spinge la coppia $\langle \text{terminale } I[j], \text{stato } k \rangle$ sullo stack. Si avanza l'indice j per leggere il prossimo simbolo di lookahead.

reduce $X \rightarrow \alpha$:

pop $|\alpha|$ pairs,

push $\langle X, \text{goto}[\text{top_state}(\text{stack}), X] \rangle$

Azione: Se la tabella indica Reduce con la regola $X \rightarrow \alpha$: Si rimuovono dallo stack un numero di coppie pari alla lunghezza di α ($|\alpha|$). Questo riporta lo stack allo stato prima che α fosse riconosciuto. Si consulta lo stato che si trova ora in cima allo stack (che chiamiamo S_{old}). Si usa la funzione GOTO sulla tabella, consultando $\text{GOTO}[S_{\text{old}}, X]$ per trovare il nuovo stato S_{new} . Si spinge la coppia $\langle \text{non-terminale } X, \text{stato } S_{\text{new}} \rangle$ sullo stack. Importante: L'indice j non viene incrementato; il lookahead rimane lo stesso per la prossima iterazione.

accept: halt normally

error: halt and report error

Questo accade tipicamente nello stato dove risiede l'item $E' \rightarrow E \backslash \text{pallino}$, quando il lookahead è $\$$ (fine input). Il parsing è completato con successo.

Se la cella della tabella di parsing è vuota, la stringa di input non è legalmente generata dalla grammatica in quella configurazione. Il parser si ferma.

ANTLR si può
usare con
qualsiasi
linguaggio.
Bison solo con C

bison: a parser generator

Generatore di Parset Bottom-Up
(LALR(1) é piú potente di SLR(1))

- **bison generates parsers building the LALR(1) automaton/table**
 - slightly more complex variant of SLR(1) such that a conflict-free table is generated for more grammars
- **bison input:**
 - file with token description, priority and associativity rules, grammar rules, and some auxiliary C intructions
- **Bison generates two files in output:**
 - a C program containing the code for the parser
 - the LALR(1) table

Contiene la logica
dell'algoritmo di parsing
Bottom-Up (lo scheletro
Shift-Reduce) e le
funzioni necessarie per
interagire con il lexer.

bison: structure of bison input

%{

C declarations

%}

Bison declarations

%%

Grammar rules

%%

Programs

bison: example of input

```
%{  
#include <stdio.h>  
int yylex() ; funzione che invoca il lexer  
void yyerror(char *s){ printf("parser error") ; } funzione che il parser invoca  
quando c'è errore sintattico  
%}
```

```
%token ID WHILE BEGIN END DO IF THEN ELSE SEMI ASSIGN
```

```
%start prog
```

```
%%
```

```
prog      : stmlist;  
stmlist   : stm  
           | stmlist SEMI stm;  
stm       : ID ASSIGN ID  
           | WHILE ID DO stm  
           | BEGIN stmlist END  
           | IF ID THEN stm  
           | IF ID THEN stm ELSE stm;
```

→ Variabile Iniziale

Grammatica

Terminali in maiuscolo,
variabili in minuscolo

bison: example of output

- In the generated table we read:

state 18 contains 1 shift/reduce conflict.

...

state 18

stm -> IF ID THEN stm . (rule 7)

stm -> IF ID THEN stm . ELSE stm (rule 8)

ELSE shift, and go to state 19

ELSE [reduce using rule 7 (stm)]

Ho conflitto Shift-Reduce
perché la grammatica é ambigua

Il puntino indica il puntino
rosso dei SLR Item

- If there are no explicit precedence declarations, BISON follows the following:
 - In shift/reduce conflicts the shift is executed
 - In reduce/reduce conflicts the first rule is applied